ComPar 26/27 Project Assignment 1 — Large-Scale Analytics with Spark

October 6, 2026

Deadline: 23:59 of November 6, 2026
Details on how to submit will be given later.

Background and Motivation

Writing correct distributed code is no longer the main bottleneck — AI tools can generate a working Spark pipeline in minutes. What remains genuinely hard, and what this assignment addresses, is making that pipeline *fast*: understanding why it is slow, forming hypotheses about the cause, applying targeted changes, and measuring whether they worked.

This requires a systematic, measurement-driven process of identifying and eliminating bottlenecks. It is distinct from optimistic tuning (changing knobs and hoping).

Spark is an industrial-strength distributed data processing engine with rich instrumentation — execution plans, DAG visualisations, stage-level metrics, and driver logs — precisely the tooling a performance engineer needs. The cluster setting introduces costs that do not exist on a single machine: data shuffles across the network, skewed partitions that stall a stage, repeated re-computation of expensive intermediate results. Every one of these has a known remedy; finding out *which* applies to *your* pipeline, quantifying its cost, and demonstrating that the remedy works is what this project asks you to do.

Datasets

The assignment uses the **NYC Taxi & Limousine Commission (TLC) Yellow Taxi Trip Records**, a large real-world dataset maintained by the City of New York. You will work with yellow cab trip data for the years **2009 to 2026** (in Parquet).
The data is available on the NYC TLC website.

The dataset has two components. **Monthly trip records** (`yellow_tripdata_YYYY-MM.parquet` , one file per month) contain the columns listed in the table below. A small **zone lookup table** (`taxi_zone_lookup.csv` , 265 rows) maps each `LocationID` to a `Borough` (Bronx, Brooklyn, Manhattan, Queens, Staten Island, EWR), a named `Zone` , and a `service_zone` (Boro Zone, Yellow Zone, Airports). Several operations require joining the zone lookup table twice — once on `PULocationID` and once on `DOLocationID` . You may start development on a single year; and if the DI cluster becomes available final process more years.

Column	Type	Description
`tpep_pickup_datetime`	timestamp	Trip start time
`tpep_dropoff_datetime`	timestamp	Trip end time
`PULocationID`	int	Pickup TLC zone ID (FK → zone lookup)
`DOLocationID`	int	Dropoff TLC zone ID (FK → zone lookup)
`passenger_count`	int	Number of passengers
`trip_distance`	double	Distance in miles
`RatecodeID`	int	1 = Standard, 2 = JFK, 3 = Newark, 4 = Nassau/Westchester, 5 = Negotiated, 6 = Group
`payment_type`	int	1 = Credit card, 2 = Cash, 3 = No charge, 4 = Dispute
`fare_amount`	double	Meter fare (USD)
`tip_amount`	double	Tip (USD)
`total_amount`	double	Total charged to passenger (USD)

Please do not push the dataset to the repository. Store it locally and document the exact download URL and version in your README.

Assignment — Pipeline Design

Your pipeline must be implemented in **PySpark** and should accept command-line parameters for at least the input data path(s) and the dataset scale (e.g. one year vs. all years, a fraction of records for smoke-testing).

What you must build

The pipeline is structured around **mandatory** and **optional** operations.
All groups must implement **M1, M2, and M3**. Additionally, each group must choose and implement **at least two** of the five optional operations (O1–O4). Your selection of optional operations should drive your performance engineering work: the four options stress different aspects of Spark execution (shuffle volume, window partition sizing, skew, UDF overhead, multi-way join ordering), and your report must connect your choice of optimisations to the specific bottlenecks each selected operation exposes.

Mandatory operations

M1 — Data ingestion and cleaning. Load the monthly Parquet files. Enforce the schema (cast types, handle nulls). Filter out invalid records on all of the following criteria:

- `fare_amount ≤ 0`,
- `trip_distance ≤ 0`,
- `passenger_count = 0` or `> 6`,
- dropoff timestamp before or equal to pickup, and `PULocationID` or `DOLocationID` outside the valid range (1–263).

Compute and report per-year cleaning statistics: count of records removed per criterion.

M2 — Temporal demand analysis. For each `(pickup_borough, day_of_week, hour_of_day)` triple, compute:

- total trip count,
- mean `fare_amount`,
- mean trip duration in minutes (derived from timestamps), and
- mean `trip_distance`.

Write the result partitioned by `borough` in Parquet format.

M3 — Zone-level statistics. For each pickup zone and month, compute mean and standard deviation of `fare_amount` and `trip_distance`. Use window functions to:

- rank zones by monthly trip volume within each borough,
- compute each zone's rolling 3-month trip share within its borough, and
- flag zones whose current-month volume deviates from their own rolling mean by more than 1.5 standard deviations.

Optional operations — choose at least 2

O1 — Fare anomaly detection. For each `(PULocationID, hour_of_day)` bucket,

- compute mean and σ of `fare_amount / trip_distance`.
- flag trips where the per-km fare exceeds mean + 3σ.
- compare weekday vs. weekend anomaly rates broken down by `payment_type` and pickup borough.

O2 — Origin-destination flow analysis.

- Compute monthly trip counts for all `(PULocationID, DOLocationID)` pairs. Join the zone lookup to resolve zone names and boroughs.
- Compute each route's share of total outbound trips from its pickup zone per month.
- Identify the top-20 routes per pickup borough and detect routes whose monthly share reverses direction between Q1 and Q3.

O3 — Demand forecasting features. For each `(PULocationID, day_of_week, hour_of_day)` bucket, compute mean historical trip count, variance, and a 4-week-lag value.

Model the lag as a self-join on offset pickup dates. Derive a demand index (current count ÷ historical mean) and flag zone-hour buckets where the demand index exceeds 2.0. Join zone lookup to annotate results by `service_zone`.

O4 — Trip-type classification. Compute, per `(PULocationID, hour_of_day)` bucket:

- fraction of airport trips ($\texttt{RatecodeID} \in \{2, 3\}$),
- negotiated fares ($\texttt{RatecodeID} = 5$), and
- group trips ($\texttt{passenger_count} \geq 4$).

Classify each bucket as `residential`, `commercial`, `transit`, or `mixed` based on these fractions and configurable thresholds. Join the classification back to the full trip table and compute fare and distance statistics per zone type per month.

In your report, describe each operation you implemented (M1–M3 and your two optional choices). For the optional operations, justify your selection from a performance engineering perspective: what bottleneck does each one expose, and why is it an interesting optimisation target?

Implementation note

You are free to use AI assistance to generate the initial implementation. The assessment does not reward implementation novelty — it rewards your ability to profile, diagnose, and improve the performance of whatever pipeline you start with. If you use AI tools to generate code, say so in the report's acknowledgements section.

Performance Engineering Phase

This is the central deliverable of the project. You must follow the loop: **profile → hypothesise → optimise → measure → repeat**. Do not apply optimisations speculatively — every change must be preceded by a profiling step that identifies the bottleneck it addresses.

Profiling and baseline

Before any optimisation, establish a **baseline**; run your full pipeline end-to-end and record:

- **on your machine:** shuffle read/write bytes, and number of tasks.
- **on the DI cluster**: wall-clock time, per-stage duration.

Use the Spark UI or the profiler of Lab3 (which you can modify at will) to inspect the DAG and stage metrics. Take screenshots where they support your analysis. You must include the baseline in your report.

Examine the execution plan with `df.explain(mode="extended")` or `df.explain(True)`. Look for:

- **SortMergeJoin** where a broadcast join would be cheaper
- **Exchange** (shuffle) nodes that dominate stage time
- **FileScan** that reads more columns or partitions than needed
- Stages with highly **skewed task durations** (one task takes 10× the median)

Scalability analysis (subject to DI cluster availability)

Run your *optimised* pipeline under two conditions and report the results:

1. **Dataset scale**: run on a 10–20% sample, then on the full dataset. Does execution time scale sub-linearly, linearly, or worse? Why?
2. **Cluster size**: run on 2 nodes, then on the full cluster allocation. Compute the speedup and efficiency. Discuss any deviation from ideal linear speedup.

You may supplement cluster measurements with runs on your personal machine (using `local[*]` mode) to isolate effects, but cluster results (at least in the Docker Spark environment) are mandatory.

Project Report

The report must read as a **performance engineering report**, not a system description. The reader should be able to follow your reasoning from observation to diagnosis to remedy to measurement. Spend at most one page on what the pipeline does; spend the remaining pages on what you measured, what you changed, and what happened.

The report is limited to **four pages** of text and graphics, plus one extra page for bibliography, acknowledgements, individual contributions, and comments. Follow the LaTeX template in the `report/` folder.

Required sections

1. Introduction — dataset choice, three analytical tasks, and a one-paragraph motivation for why this workload is interesting from a performance standpoint.

2. Pipeline — a brief description of the pipeline stages and the architectural decisions made (e.g. why you chose DataFrame API vs. Spark SQL for a given task). Include the physical execution plan (or a simplified diagram) of at least one query.

3. Profiling and Baseline — the experimental setup (hardware, Spark version, cluster size), the baseline measurements, and the bottlenecks identified from the Spark UI and execution plans. Support claims with evidence — numbers, screenshots, or annotated DAGs.

4. Optimisation Study — one subsection per optimisation applied. Each subsection must state: (a) the bottleneck observed, (b) the hypothesis, © the change made, and (d) the measured before/after result. A table summarising all optimisations and their impact is required at the end of this section.

5. Analysis — results for dataset scale and cluster size, with discussion. Include at least one chart (speedup curve or execution time vs. dataset fraction).

6. Discussion — remaining bottlenecks not addressed, limitations of the methodology, and what you would do differently with more time.

Reproducibility

All experiments must be reproducible. Document: Spark version, cluster topology (number of workers, cores, memory per worker), `spark-submit` command, and any non-default Spark configuration properties. A `README.md` with exact reproduction steps is required.

Extra page

Section	Content
Bibliography	All references cited in the text
Acknowledgements	Colleagues who helped, AI tools used (required if any), and how they were used
Individual contributions	Work division and percentage per member
Comments and suggestions	Free-form feedback (no impact on grade)

Submission and Grading

Grading criteria

Group grade (G) is primarily based on the written report and reflects the quality of the performance engineering analysis. Additional criteria: ease of compilation and execution, code cleanliness, correctness of results, and the rigour and impact of the optimisations demonstrated.

Individual grade (S) may vary among group members and is determined by the group's reported work division and the instructor's assessment of each member's contribution as evident from the report.

The individual lab grade is calculated as:

```
0.5*G + 0.5*S
```

Both the group members and the instructor may request an individual or group presentation. Grades may be revised upwards or downwards following such a presentation.

What makes a strong report

- Every optimisation is backed by a *before* measurement that identifies the bottleneck, not just an *after* measurement that shows improvement.
- Claims about the cause of slowness are supported by Spark UI evidence (stage durations, shuffle bytes, skew indicators).
- The scalability analysis interprets deviations from ideal speedup rather than just reporting numbers.
- The writing is precise and concise: numbers have units, comparisons have baselines, and conclusions follow from data.